CS1234 — Small-Scale Application Development (Linux + tooling)

Tier	A
Phase (order)	1
Do this first	CS1111 (basic C)
Estimated effort	4-6 weeks, ~5 h/week
Unlocks	Everything later: you will use git, make, gdb, scripts daily

Roadmap

1. Learn the shell properly: files, pipes, grep/sed/awk, bash scripts.
2. Learn git end-to-end: commit, branch, merge, rebase, remote.
3. Learn make + gdb + valgrind by building a multi-file C program (headers, separate compilation).
4. Write 3-4 small utilities (word count, log parser, file organiser, a simple linked-list app) and one Python script version of each.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Missing Semester IAP 2020 (11 videos). MIT *Missing Semester* (2020): shell, vim, scripting, git, debugging, Makefiles. Do every exercise.

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- Missing Semester IAP 2026 (9 videos). Missing Semester IAP 2026: the refreshed edition.
- Linux Commands for Beginners (25 videos). Learn Linux TV *Linux Commands for Beginners*.

⚡ Short videos for intuition and revision

- Linux for Hackers (and everyone) // FREE Course for Beginners (10 videos). NetworkChuck *Linux for Hackers*: fun, motivating, short.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–2	Shell + shell scripting. Write a script that renames/organises files in a folder.

Week	Lectures	What to do
2	3–5	Vim, data wrangling (grep/sed/awk/pipes), job control, tmux, dotfiles. Put your dotfiles in a git repo.
3	6	Git: watch it twice. Then do learngitbranching.js.org fully.
4	7–10	Debugging/profiling (gdb, perf), Makefiles, security basics. Build a small C or Python CLI with a Makefile, tests and a README on GitHub.

▼ **All lectures in *Missing Semester IAP 2020* (11 videos)**

1. Lecture 1: Course Overview + The Shell (2020)
2. Lecture 2: Shell Tools and Scripting (2020)
3. Lecture 3: Editors (Vim) (2020)
4. Lecture 4: Data Wrangling (2020)
5. Lecture 5: Command-line Environment (2020)
6. Lecture 6: Version Control (git) (2020)
7. Lecture 7: Debugging and Profiling (2020)
8. Lecture 8: Metaprogramming (2020)
9. Lecture 9: Security and Cryptography (2020)
10. Lecture 10: Potpourri (2020)
11. Lecture 11: Q&A (2020)

Books, notes and websites

- MIT *The Missing Semester of Your CS Education* — missing.csail.mit.edu
- Pro Git (free) — git-scm.com/book
- Beej's Guide to C and Beej's Guide to Unix IPC
- Book: *Automate the Boring Stuff with Python* (free online) for the Python part

Practice / proof of understanding

A GitHub repo with 4 utilities, a Makefile, a README and at least 20 meaningful commits.

Note

Do this in the first month; it makes every later lab easier.

What your IITM course sheet says

From the PDF: CS1234

Official description: The high-level philosophy is to develop systemic skills and help make our students become more productive in problem solving. The emphasis is on developing various applications useful for small-scale processing. Each application would benefit from a set of programming and helper tools, which would be taught and practiced in this course

Official syllabus (trimmed):

> The course covers programming and tools via the development of multiple small-scale applications. Each such application or utility can be coded more effectively by combining programming tools along with the available operating system tools. The development would happen for one or two weeks per application. The course is planned with primarily C programming, Linux tools, and introduces Python. Each application would take 1 to 2 weeks of discussion. Programming Constructs, Techniques, and Problems: Recall: Pointers, Structures, Dynamic Memory Allocation, File I/O. Working with multiple files (multiple headers, multiple code files): combined with editor control, version control, scripting. File Handling: more problems combined with scripting Linked Lists: Single, Double, Circular, Stack, Queue along with applications Recursion: advanced problems (such as backtracking) Arrays (multi-dimensional): advanced problems with linear and tabular data Bitwise Operators Command-line Arguments (combined with testing) Using libraries (such as Maths, Geometry, Matrices, Graphics) Python: basic constructs and libraries Tools: Editors, Makefile, Testing, Debugging (such as gdb), Version Control (github), Profiling (measuring time and memory), Shell Scripting (such as bash and utilities such as grep, sed, awk, tar, zip, find), Operational Basics (ssh, scp, wget, rsync), Security

Official textbooks: 1. Kernighan and Pike, The Practice of Programming, 1st Edition, Addison-Wesley Professional Computing Series, 1999 2. Ebrahim and Mallett, Mastering Linux Shell Scripting - Second Edition: A practical guide to Linux command-line, Packt Publishers, 2018

Official references: 1. Robert Love, Linux System Programming: Talking Directly to the Kernel and C Library, Second Edition, O'Reilly, 2014 2. Expert C Programming (Deep C Secrets), Third edition, Peter Van Der Linden, Pearson Education, 2021 3. Python: The Complete Reference, Martin C. Brown, Mc Graw Hill, 2018